

FIAP

NBA

BANCOS DE DADOS COLUNARES E TIMESERIES





MARCELO BARBOSA

Engenheiro de software com 20 anos de mercado.

Análise de Sistemas da Informação pela Universidade Estácio de Sá.

MBA em Big Data pela FIAP.

Passagens por empresas como B2W, Itaú e Santander.

Esposo da Luciana e pai da Ana Júlia e do Vítor.

Curte família, comida, tecnologia e música.

- LinkedIn: infobarbosa
- Github: infobarbosa

INTRODUÇÃO



Oferecer ao aluno uma visão consistente e prática da linguagem CQL (Cassandra Query Language) assim como uma robusta metodologia de modelagem de dados para Apache Cassandra.



CQL

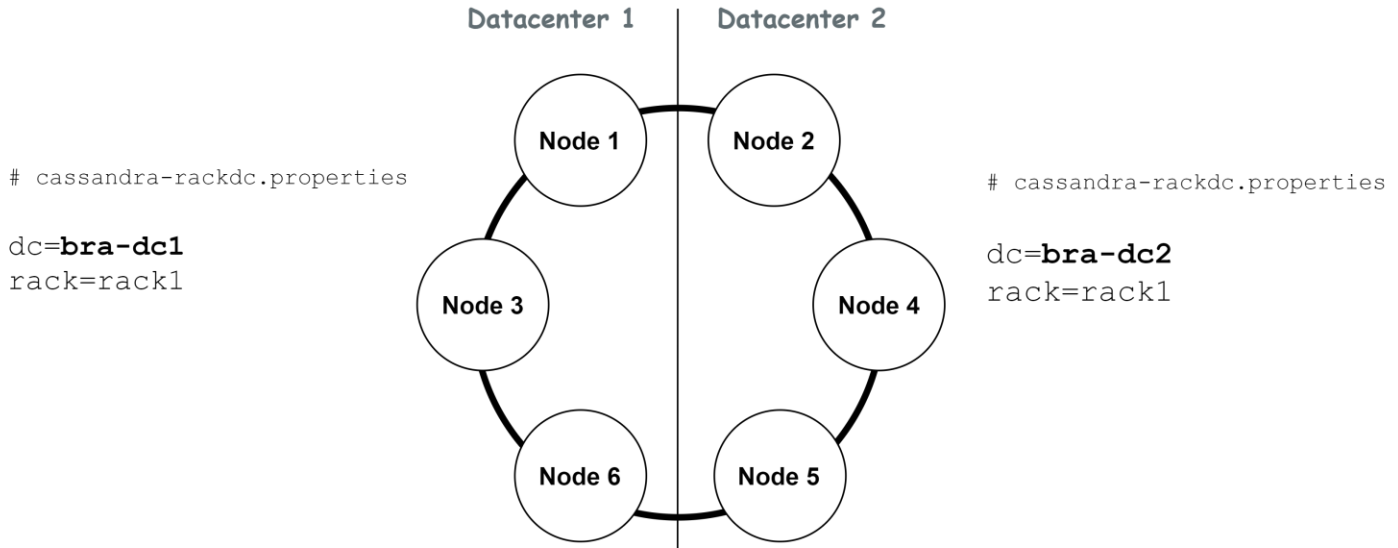
CQL é uma linguagem declarativa desenvolvida para prover abstração no acesso de aplicações/usuários ao Apache Cassandra.

CREATE KEYSPACE

```
CREATE KEYSPACE [IF NOT EXISTS] keyspace_name
WITH REPLICATION = {
    'class' : 'SimpleStrategy', 'replication_factor' : N }
| 'class' : 'NetworkTopologyStrategy',
  'dc1_name' : N [, ...]
}
[AND DURABLE_WRITES = true|false] ;
```


CQL – CREATE KEYSPACE

CREATE KEYSPACE – Exemplo



Exemplo:

```
CREATE KEYSPACE infobarbank
WITH REPLICATION = {
  'class' : 'NetworkTopologyStrategy',
  'AUST' : 3 , // São Paulo/Brasil/AZ1
  'BRA' : 6 , // São Paulo/Brasil/AZ2
  'CAN' : 3 , // São Paulo/Brasil/AZ2
};
```

CREATE TABLE

```
CREATE TABLE [IF NOT EXISTS] [keyspace_name.]table_name (  
    column_definition [, ...]  
    PRIMARY KEY (column_name [, column_name ...])  
[WITH table_options  
    | CLUSTERING ORDER BY (clustering_column_name order)]  
    | ID = 'table_hash_tag'  
    | COMPACT STORAGE]
```

Exemplo:

```
CREATE TABLE infobarbank.cliente(  
    id      uuid PRIMARY KEY,  
    cpf     text,  
    nome    text  
);
```

```
CREATE TABLE infobarbank.cliente(  
    id      uuid,  
    cpf     text,  
    nome    text,  
    PRIMARY KEY(id)  
);
```

CQL – CREATE TABLE

Partição de linha única

CLIENTE							
ID	NOME	CPF	EMAIL	TELEFONE	DATA_NASC	RENDIA	UF
1	MARIVALDA KANAMARY	987.539.360-60	marivalda@bmail.com	(11) 987654321	06/02/1997	R\$5.000,00	SP
2	JUCILENE MOREIRA CRUZ	124.554.260-50	jucilene@bmail.com	(21) 945678900	06/02/1944	R\$2.500,00	RJ

```
CREATE TABLE infobarbank.cliente(  
  id uuid,  
  cpf text,  
  nome text,  
  ...  
  PRIMARY KEY(id)  
);
```

Column Family Cliente

row key 1 hash(id=1)	<table><tr><td>nome</td></tr><tr><td>MARIVALDA ...</td></tr><tr><td>1660150830</td></tr></table>	nome	MARIVALDA ...	1660150830	<table><tr><td>cpf</td></tr><tr><td>987.539.360-60</td></tr><tr><td>1660151879</td></tr></table>	cpf	987.539.360-60	1660151879	<table><tr><td>email</td></tr><tr><td>marivalda@gmail.com</td></tr><tr><td>1660141001</td></tr></table>	email	marivalda@gmail.com	1660141001	<table><tr><td>...</td></tr><tr><td>...</td></tr><tr><td>...</td></tr></table>	...	...	...
nome																
MARIVALDA ...																
1660150830																
cpf																
987.539.360-60																
1660151879																
email																
marivalda@gmail.com																
1660141001																
...																
...																
...																
row key 2 hash(id=2)	<table><tr><td>nome</td></tr><tr><td>JUCILENE ...</td></tr><tr><td>1660151879</td></tr></table>	nome	JUCILENE ...	1660151879	<table><tr><td>cpf</td></tr><tr><td>124.554.260-50</td></tr><tr><td>1660151960</td></tr></table>	cpf	124.554.260-50	1660151960	<table><tr><td>data_nasc</td></tr><tr><td>06/02/1944</td></tr><tr><td>1660151981</td></tr></table>	data_nasc	06/02/1944	1660151981	<table><tr><td>...</td></tr><tr><td>...</td></tr><tr><td>...</td></tr></table>	...	...	...
nome																
JUCILENE ...																
1660151879																
cpf																
124.554.260-50																
1660151960																
data_nasc																
06/02/1944																
1660151981																
...																
...																
...																
row key 7 hash(id=7)	<table><tr><td>nome</td></tr><tr><td>LUCILIA ...</td></tr><tr><td>1660151879</td></tr></table>	nome	LUCILIA ...	1660151879		<table><tr><td>email</td></tr><tr><td>lucilia@yahoo.com</td></tr><tr><td>1660151981</td></tr></table>	email	lucilia@yahoo.com	1660151981	<table><tr><td>...</td></tr><tr><td>...</td></tr><tr><td>...</td></tr></table>	...	...	...			
nome																
LUCILIA ...																
1660151879																
email																
lucilia@yahoo.com																
1660151981																
...																
...																
...																

Chave composta

Ocorre quando a **chave primária** consiste em mais de uma coluna

```
PRIMARY KEY (partition_column_name, clustering_column_name [, ...])
```

Exemplo:

```
CREATE TABLE infobarbank.cartao_credito(  
    id_cliente      INT,  
    id_cartao       UUID,  
    ds_tipo         TEXT, //FISICO ou VIRTUAL  
    nm_cartao       TEXT,  
    ds_numero       TEXT,  
    nr_cvv          TEXT,  
    dt_emissao      DATE,  
    dt_validade     DATE,  
    vl_limite       FLOAT,  
    vl_saldo        FLOAT,  
    PRIMARY KEY(id_cliente, id_cartao)  
);
```

CQL – CREATE TABLE

Chave composta

Partição de múltiplas linhas

Exemplo:

CARTAO_CREDITO				
id_cliente	id_cartao	ds_tipo	ds_numero	...
1	007c2c1c	FISICO	... 8454	...
1	897199aa	FISICO	... 7490	...
1	9a748c3c	VIRTUAL	... 8850	...
2	a2fab366	FISICO	... 1778	...
7	cd586ffb	VIRTUAL	... 3738	...

CQL – CREATE TABLE

Chave composta

Partição de múltiplas linhas

```
CREATE TABLE infobarbank.cartao_credito (
  id_cliente      INT,
  id_cartao       UUID,
  ds_tipo         TEXT, //FISICO ou VIRTUAL
  nm_cartao       TEXT,
  ds_numero       TEXT,
  nr_cvv          TEXT,
  dt_emissao      DATE,
  dt_validade     DATE,
  vl_limite       FLOAT,
  vl_saldo        FLOAT,
  PRIMARY KEY(id_cliente, id_cartao)
);
```

Partition key: id_cliente
Clustering column: id_cartao

CARTAO_CREDITO				
id_cliente	id_cartao	ds_tipo	ds_numero	...
1	007c2c1c	FISICO	... 8454	...
1	897199aa	FISICO	... 7490	...
1	9a748c3c	VIRTUAL	... 8850	...
2	a2fab366	FISICO	... 1778	...
7	cd586ffb	VIRTUAL	... 3738	...

Column Family CARTAO_CREDITO

row key 1 hash(id_cliente=1)	id_cartao 007c2c1c	ds_tipo FISICO	ds_numero ... 8454	...
	id_cartao 897199aa	ds_tipo FISICO	ds_numero ... 7490	...
	id_cartao 9a748c3c	ds_tipo VIRTUAL	ds_numero ... 8850	...
row key 2 hash(id_cliente=2)	id_cartao a2fab366	ds_tipo FISICO	ds_numero ... 8454	...
row key 7 hash(id_cliente=7)	id_cartao cd586ffb	ds_tipo VIRTUAL	ds_numero ... 8454	...

CQL – CREATE TABLE

Chave composta

Partição de múltiplas linhas

```
CREATE TABLE infobarbank.cartao_credito(  
    id_cliente      INT,  
    id_cartao       UUID,  
    ...  
    PRIMARY KEY(id_cliente)  
);
```



Nesse caso a tabela seria criada com uma única partição e não permite múltiplas linhas

```
CREATE TABLE infobarbank.cartao_credito(  
    id_cliente      INT,  
    id_cartao       UUID,  
    ...  
    PRIMARY KEY(id_cliente, id_cartao)  
);
```



CQL – CREATE TABLE

Chave composta

Chave de partição de múltiplas colunas

Column Family **LANCAMENTO_CARTAO_CREDITO**

row key 1000 <code>hash (id_cliente=1, dt_competencia=202207)</code>	id_cartao	ts_trans	ds_trans	...
	007c2c1c	2022-07-08 09:30:26.123	Mister Donuts	...
	id_cartao	ts_trans	ds_trans	...
	007c2c1c	2022-07-09 10:49:00.127	Posto Canario	...
	id_cartao	ts_trans	ds_trans	...
	007c2c1c	2022-07-10 20:00:10.000	Mambo Supermerc	...
row key 1001 <code>hash(id_cliente=1,dt_competencia=202208)</code>	id_cartao	ts_trans	ds_trans	...
	007c2c1c	2022-08-09 10:30:00.555	Posto Canario	...
row key 1050 <code>hash(id_cliente=7,dt_competencia=202207)</code>	id_cartao	ts_trans	ds_trans	...
	cd586ffb	2022-07-23 22:20:00.050	Posto Canario	...

Partition key

Clustering
columns

Primary key

CQL – CREATE TABLE

Chave composta

Chave de **partição** de **múltiplas colunas**

Ocorre quando a **partition key** consiste em mais de uma coluna

```
PRIMARY KEY (  
    (partition_column_1, partition_column_2, ...), clustering_column_1 [, ...]  
)
```

Exemplo:

```
CREATE TABLE infobarbank.lancamentos_cartao_credito(  
    id_cliente          UUID,  
    dt_competencia      TEXT,  
    id_cartao           UUID,  
    ts_trans            TIMESTAMP,  
    status              TEXT,  
    id_trans            UUID,  
    descricao           TEXT,  
    estabelecimento     TEXT,  
    valor               FLOAT,  
    PRIMARY KEY((id_cliente, dt_competencia), id_cartao, ts_trans, id_trans)  
);
```

Reflita sobre a modelagem de dados

CQL – CREATE TABLE

Chave composta – UNICIDADE

```
CREATE TABLE infobarbank.lancamentos_cartao_credito(  
    id_cliente      UUID,  
    id_cartao       UUID,  
    ts_trans        TIMESTAMP,  
    status          TEXT,  
    id_trans        UUID,  
    ...  
    PRIMARY KEY((id_cliente), id_cartao)  
);
```



```
CREATE TABLE infobarbank.lancamentos_cartao_credito(  
    id_cliente      UUID,  
    id_cartao       UUID,  
    ts_trans        TIMESTAMP,  
    status          TEXT,  
    id_trans        UUID,  
    ...  
    PRIMARY KEY((id_cliente), id_cartao, id_trans)  
);
```



CQL – CREATE TABLE

Chave composta – TAMANHO DA PARTIÇÃO

```
CREATE TABLE infobarbank.lancamentos_cartao_credito(
  id_cliente      UUID,
  id_cartao       UUID,
  ts_trans        TIMESTAMP,
  status          TEXT,
  id_trans        UUID,
  ...
  PRIMARY KEY((id_cliente), id_cartao, ts_trans, id_trans)
);
```



```
CREATE TABLE infobarbank.lancamentos_cartao_credito(
  id_cliente      UUID,
  dt_competencia  TEXT,
  id_cartao       UUID,
  ts_trans        TIMESTAMP,
  status          TEXT,
  id_trans        UUID,
  descricao       TEXT,
  estabelecimento TEXT,
  valor           FLOAT,
  PRIMARY KEY((id_cliente, dt_competencia), id_cartao, ts_trans, id_trans)
);
```



CQL – CREATE TABLE

Chave composta – ORDENAÇÃO

```
CREATE TABLE infobarbank.lancamentos_cartao_credito(  
    id_cliente      UUID,  
    id_cartao       UUID,  
    ts_trans        TIMESTAMP,  
    status          TEXT,  
    id_trans        UUID,  
    ...  
    PRIMARY KEY((id_cliente), id_cartao, id_trans)  
);
```



```
CREATE TABLE infobarbank.lancamentos_cartao_credito(  
    id_cliente      UUID,  
    id_cartao       UUID,  
    ts_trans        TIMESTAMP,  
    status          TEXT,  
    id_trans        UUID,  
    ...  
    PRIMARY KEY((id_cliente), id_cartao, ts_trans, id_trans)  
);
```



Big Partitions...

✓ Big partitions

For Cassandra it's recommended to keep the size of partitions under 100MB size. Usually the presences of the big partitions is a sign of incorrect data model, triggered by these factors:

- Low cardinality of partition keys. - when too few possible values exist for partition keys.
- Non-uniform spread of data between partitions.

For example, if a customer ID is used as a partition key, large customers applications will write much more data than smaller customers. As a result, some nodes may have much more data than other nodes. More data increase the load on these nodes because they handle more requests, require more compaction, and so on.

- Too many columns and rows in the table, especially when every row contains data for all or most columns.
- Storing big blobs or long texts in the table.
- [Big partitions](#)  create an additional load on Cassandra, such as allocation of additional memory to hold partition index.

Note: Before Cassandra versions 3.6, reading big partitions put much more pressure on Java heap and often led to nodes crashing.

- Uneven data distribution between the nodes can lead to hotspots when some nodes handle many more requests than others.
- Big partitions require transferring more data when performing reading of the whole partition.

CASSANDRA QUERY LANGUAGE

```
SELECT * | select_expression | DISTINCT partition
FROM [keyspace_name.] table_name
[WHERE partition_value
  [AND clustering_filters
  [AND static_filters]]]
[ORDER BY PK_column_name ASC|DESC]
[LIMIT N]
[ALLOW FILTERING]
```

CQL – SELECT

Exemplos:

```
SELECT *  
FROM lançamento_cartao_credito...
```

```
WHERE id_cliente = 1  
AND dt_competencia = '202208';
```

```
WHERE id_cliente = 1  
AND dt_competencia = '202208'  
AND id_cartao = '007c2c1c';
```

```
WHERE id_cliente = 1  
AND dt_competencia = '202208'  
AND id_cartao = '007c2c1c'  
AND ts_trans > '2022-07-08 09:30:26.123'  
ORDER BY ts_trans DESC;
```

```
CREATE TABLE infobarbank.lancamentos_cartao_credito(  
  id_cliente      UUID,  
  dt_competencia  TEXT,  
  id_cartao       UUID,  
  ts_trans        TIMESTAMP,  
  status          TEXT,  
  id_trans        UUID,  
  descricao       TEXT,  
  estabelecimento TEXT,  
  valor          FLOAT,  
  PRIMARY KEY(id_cliente, dt_competencia), id_cartao, ts_trans, id_trans)  
);
```



Exemplos:

```
SELECT *  
FROM lançamento_cartao_credito...
```

```
WHERE id_cliente = 1; //partition key incompleta
```

```
WHERE dt_competencia = '202208' //partition key incompleta  
AND id_cartao = '007c2c1c';
```

```
WHERE id_cliente = 1  
AND dt_competencia > '202208'; //apenas critério de igualdade
```

```
WHERE id_cliente = 1  
AND dt_competencia = '202208'  
AND id_cartao = '007c2c1c'  
AND ds_trans = 'Posto Canario'; //data column
```



```
INSERT INTO [keyspace_name.] table_name (column_list)
VALUES (column_values)
[IF NOT EXISTS]
[USING TTL seconds | TIMESTAMP epoch_in_microseconds]
```

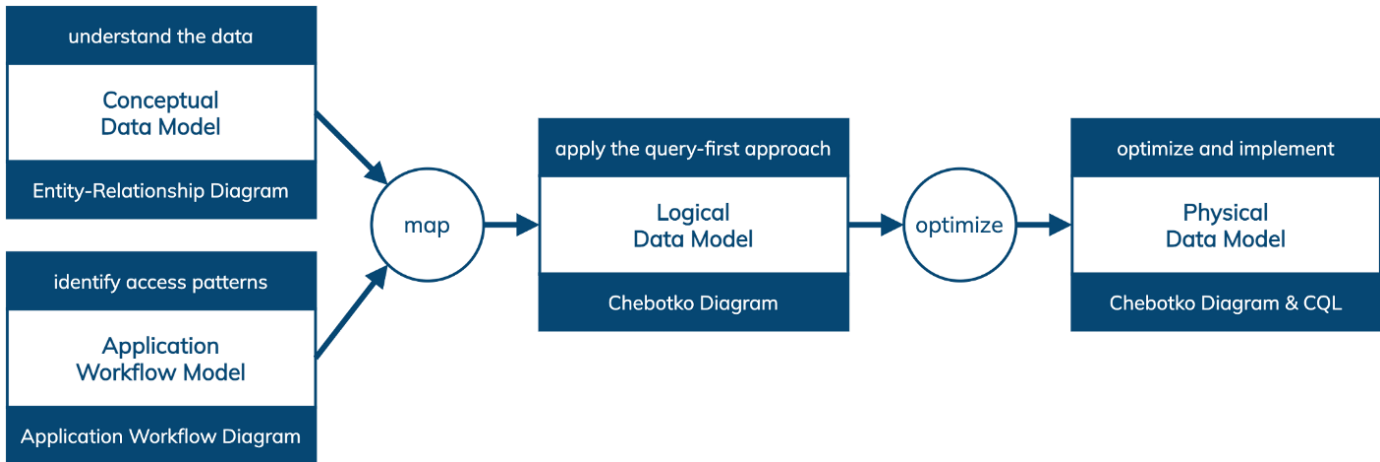
```
UPDATE [keyspace_name.] table_name  
[USING TTL time_value | USING TIMESTAMP timestamp_value]  
SET assignment [, assignment] . . .  
WHERE row_specification  
[IF EXISTS | IF condition [AND condition] . . .] ;
```

```
DELETE [column_name (term)][, ...]  
FROM [keyspace_name.] table_name  
[USING TIMESTAMP timestamp_value]  
WHERE PK_column_conditions  
[IF EXISTS | IF static_column_conditions]
```



Metodologia

Metodologia



Metodologia

Domínio de negócio

Modelo conceitual

Entendimento do dado.

Identificação de entidades chave no domínio e seus relacionamentos.

Também conhecido como **Modelo Conceitual**.

Aplicação

Fluxo aplicativo

Mapeamento do fluxo aplicativo e seus padrões de acesso (consultas), questões que queremos que o banco de dados responda.

Modelo lógico

Definição do schema de banco de dados.

Menor atenção a tipos de dados e índices.

Modelo físico

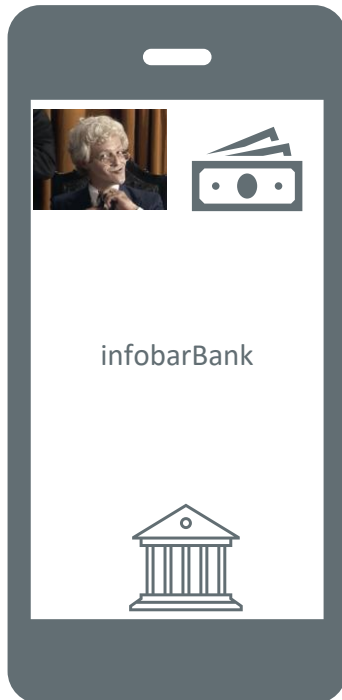
Definição do schema físico de banco de dados.

Atenção a tipos de dados, índices e otimizações necessárias.

MODELAGEM NÃO RELACIONAL

Exemplo

Um caso hipotético, o aplicativo **infobarBank**



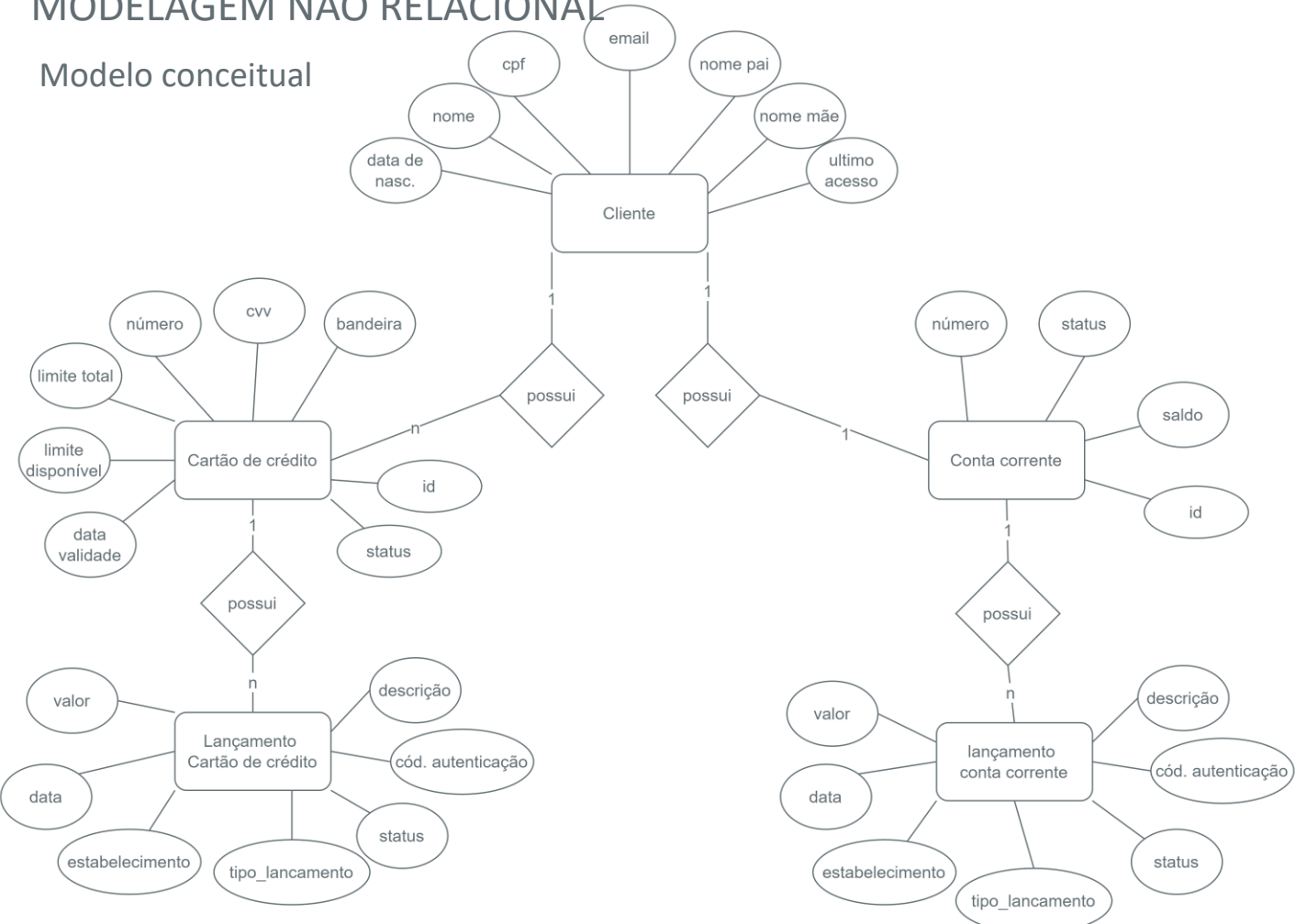
Features:

- Login
- Saldo de conta corrente
- Saldo de cartão de crédito
- Últimos lançamentos
- Transferências
- Pagamentos
- ...

Modelo conceitual

MODELAGEM NÃO RELACIONAL

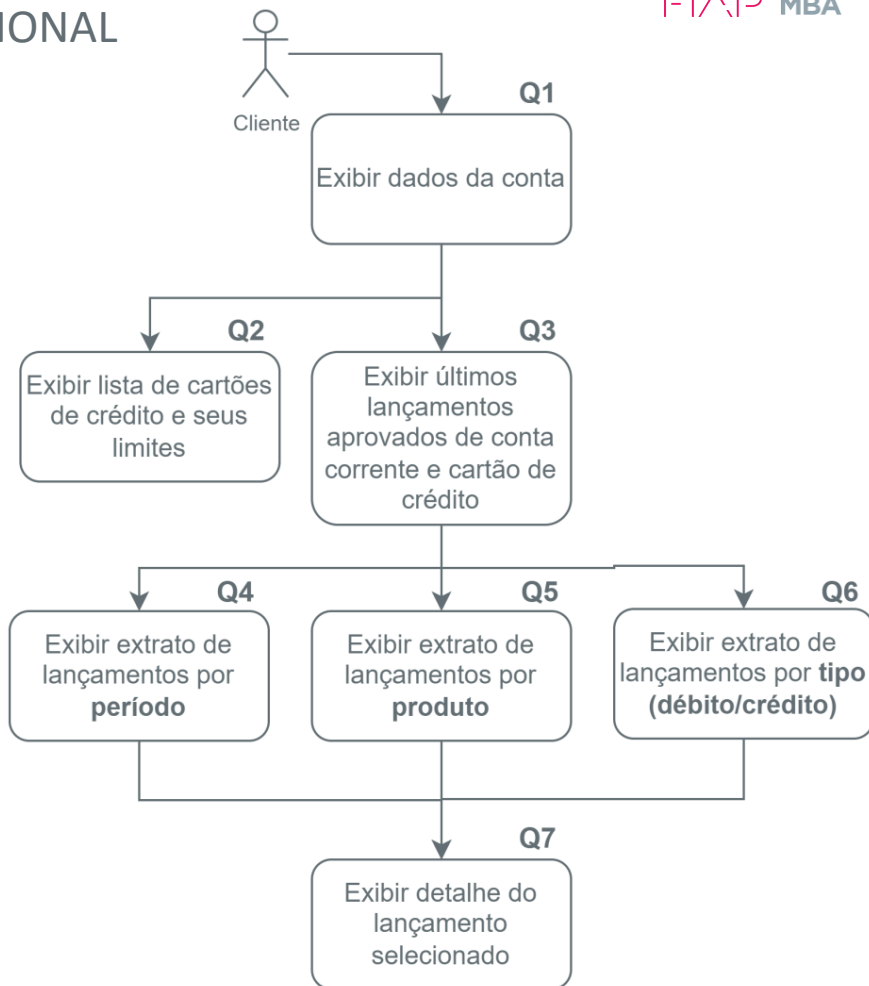
Modelo conceitual



Fluxo aplicativo

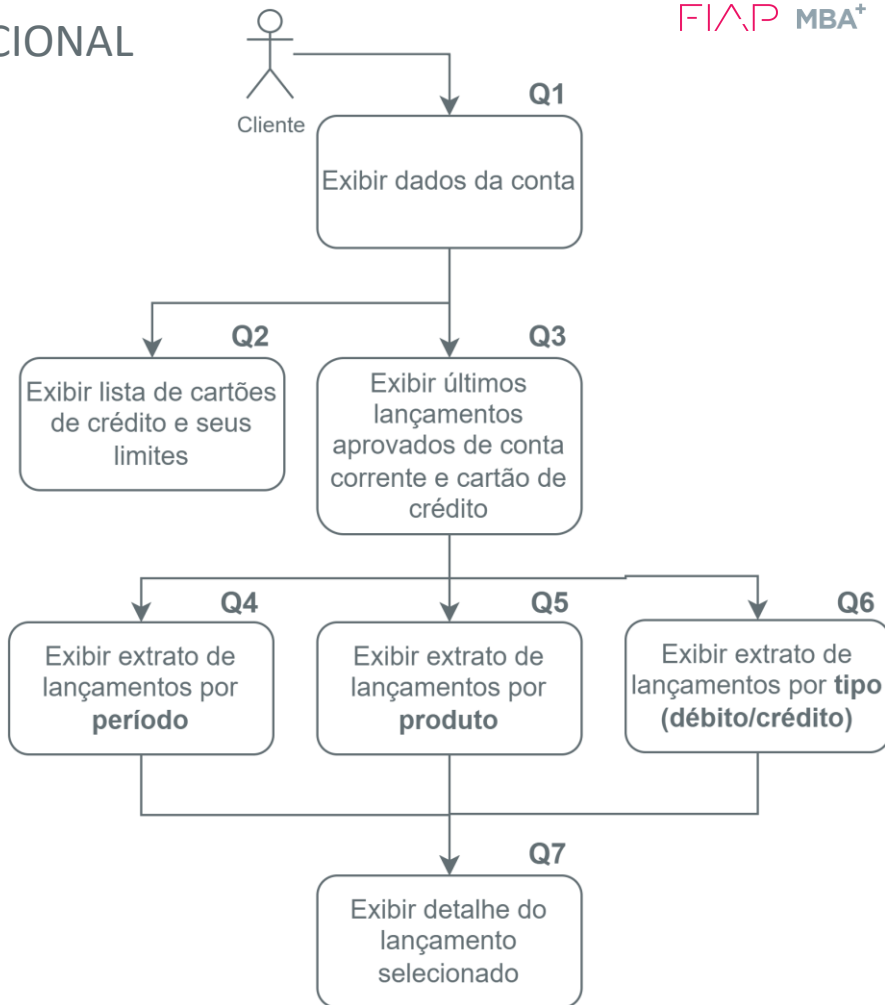
MODELAGEM NÃO RELACIONAL

Fluxo aplicacional



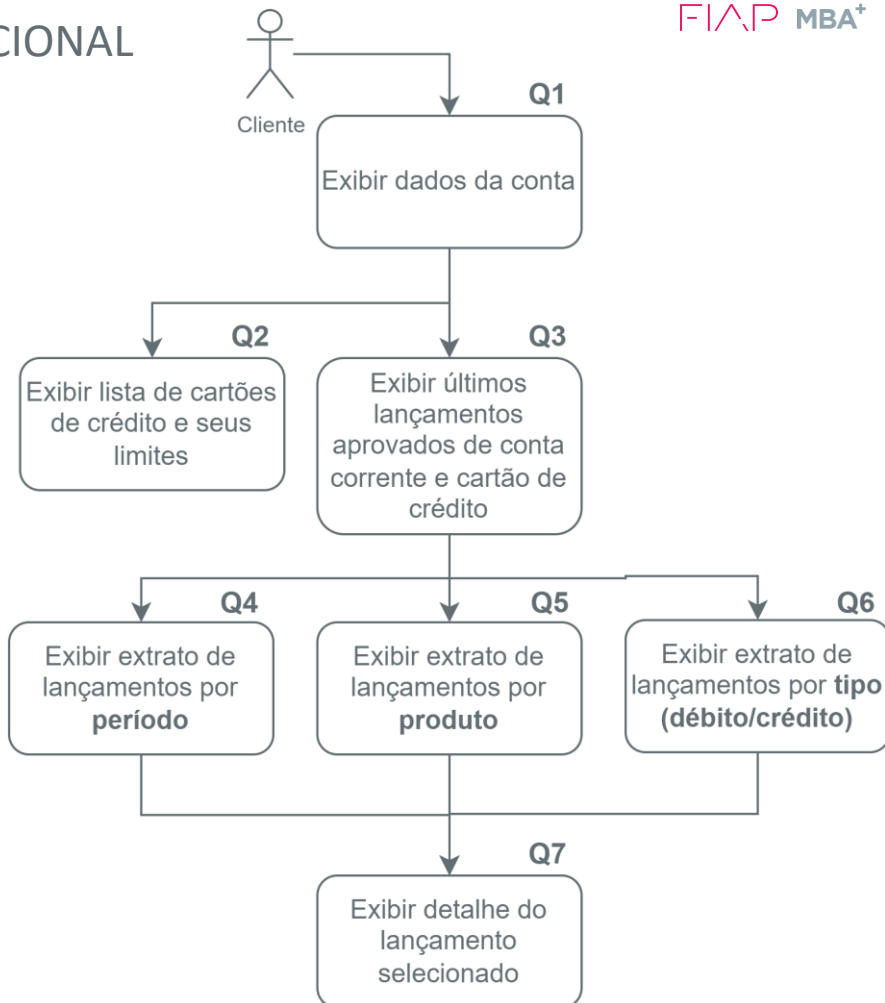
MODELAGEM NÃO RELACIONAL

Fluxo aplicacional



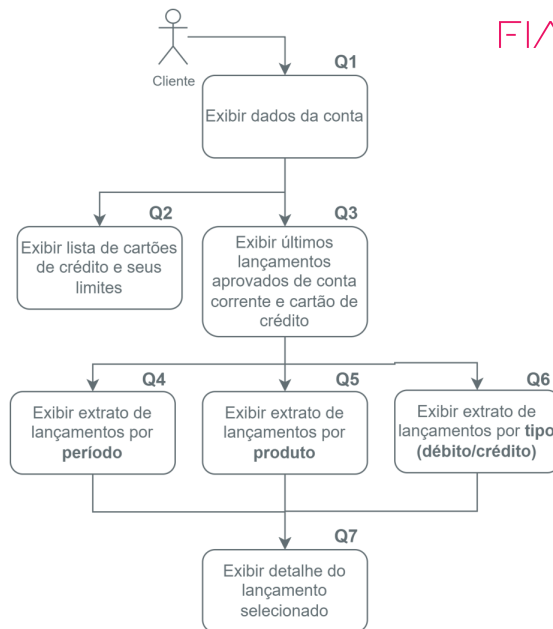
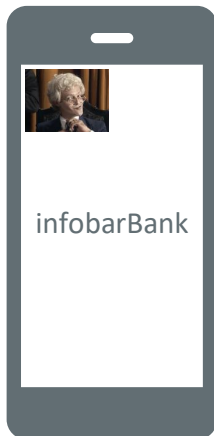
MODELAGEM NÃO RELACIONAL

Fluxo aplicacional



MODELAGEM NÃO RELACIONAL

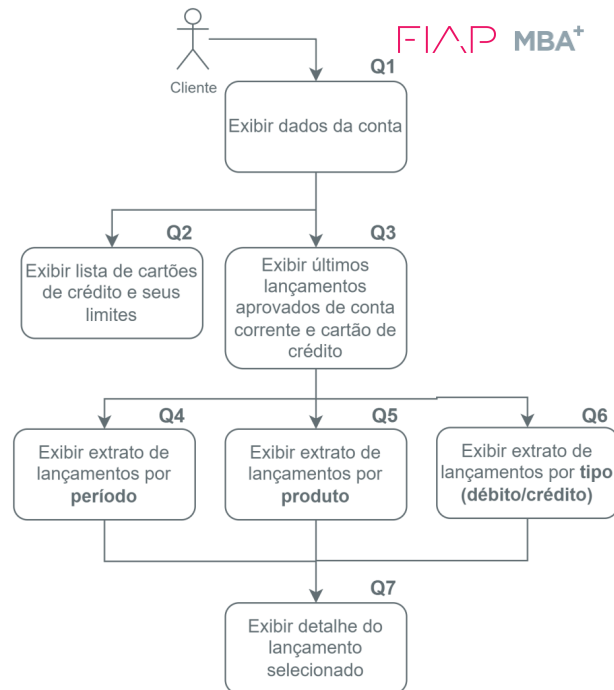
Fluxo aplicacional



- Q1:** Buscar os dados da conta do cliente (apelido do cliente, número da conta, último acesso e saldo disponível);
- Q2:** Buscar a lista de cartões de crédito ativos;
- Q3:** Buscar os últimos 3 lançamentos aprovados de conta corrente e cartão de crédito em ordem decrescente de data.
- Q3.1:** Caso o cliente não tenha lançamentos no mês corrente, buscar as três últimas do mês anterior e assim sucessivamente.
- Q4:** Buscar as últimas 2 semanas de lançamentos entre conta corrente e cartão de crédito em ordem decrescente de data;
- Q4.1:** Caso o cliente role a tela, buscar outras duas semanas de lançamentos;
- Q4.2:** Caso o cliente selecione apenas o produto conta corrente, buscar apenas este produto;
- Q4.3:** Caso o cliente selecione apenas um cartão de crédito, buscar apenas este produto;
- Q4.4:** Caso o cliente selecione um período e produto, buscar lançamentos utilizando esse critério;
- Q4.5:** Caso o cliente selecione apenas um tipo de lançamento (entrada ou saída), buscar movimentações utilizando esse critério;
- Q5:** Buscar o detalhe de um lançamento selecionado pelo cliente;

MODELAGEM NÃO RELACIONAL

Fluxo aplicacional

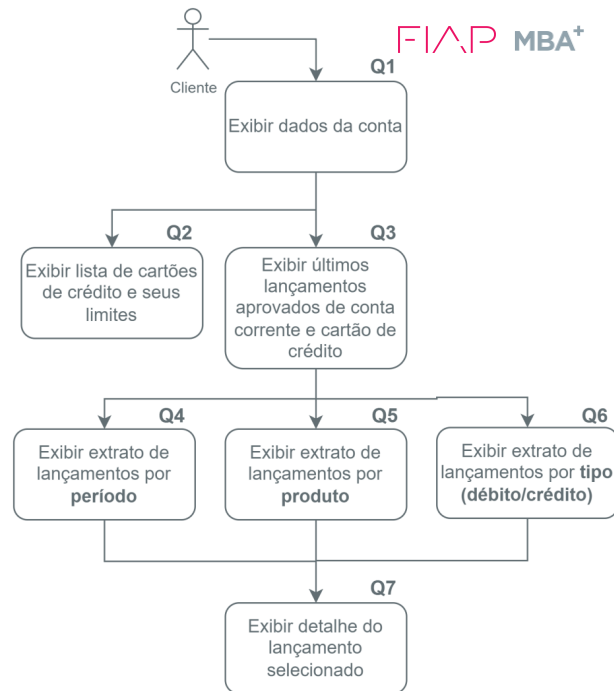
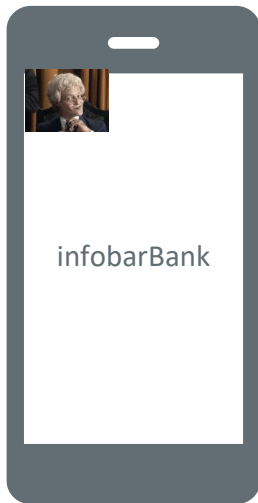


- Q1:** Buscar os dados da conta do cliente (apelido do cliente, número da conta, último acesso e saldo disponível);
- Q2:** Buscar a lista de cartões de crédito (número do cartão e limite disponível);
- Q3:** Buscar os últimos 3 lançamentos aprovados de conta corrente e cartão de crédito em ordem decrescente de data.
- Q3.1:** Caso o cliente não tenha lançamentos no mês corrente, buscar as três últimas do mês anterior e assim sucessivamente.
- Q4:** Buscar as últimas 2 semanas de lançamentos entre conta corrente e cartão de crédito em ordem decrescente de data;
- Q4.1:** Caso o cliente role a tela, buscar outras duas semanas de lançamentos;
- Q5:** Buscar os lançamentos do produto e período selecionado pelo cliente;
- Q6:** Buscar os lançamentos do tipo (crédito ou débito) e período selecionados pelo cliente;
- Q7:** Buscar o detalhe de um lançamento selecionado pelo cliente;



MODELAGEM NÃO RELACIONAL

Fluxo aplicativo



- Q1:** Buscar os dados da conta do cliente (apelido do cliente, número da conta, último acesso e saldo disponível);
- Q2:** Buscar a lista de cartões de crédito (número do cartão e limite disponível);
- Q3:** Buscar os últimos 3 lançamentos aprovados de conta corrente e cartão de crédito em ordem decrescente de data.
- Q3.1:** Caso o cliente não tenha lançamentos no mês corrente, buscar as três últimas do mês anterior e assim sucessivamente.
- Q4:** Buscar as últimas 2 semanas de lançamentos entre conta corrente e cartão de crédito em ordem decrescente de data;
- Q4.1:** Caso o cliente role a tela, buscar outras duas semanas de lançamentos;
- Q5:** Buscar os lançamentos do produto e período selecionado pelo cliente;
- Q6:** Buscar os lançamentos do tipo (crédito ou débito) e período selecionados pelo cliente;
- Q7:** Buscar o detalhe de um lançamento selecionado pelo cliente;

Modelo lógico

Lembrete

Unicidade

Ordenação

Distribuição (tamanho partição)

Modelo lógico

Q1: Buscar os dados da conta do cliente (apelido do cliente, número da conta, último acesso e saldo disponível);

conta_corrente	
num_conta	K
apelido_cliente	
data_ultimo_acesso	
saldo_disponivel	

Modelo lógico

Q2: Buscar a lista de cartões de crédito ativos;

cartao_credito	
id_cliente	K
status	C↑
id	C↑
num_cartao	
cvv	
data_validade	
limite_total	
limite_disponivel	

Modelo lógico

Q3: Buscar as últimos **3 lançamentos aprovados** de conta corrente e cartão de crédito em ordem decrescente de data.

Q3.1: Caso o cliente não tenha lançamentos no mês corrente, buscar as três últimas do mês anterior e assim sucessivamente.

lançamentos_por_status	
id_cliente	K
status	C↑
data_hora	C↓
id	C↑
id_produto	
tipo_lancamento	
descricao	
estabelecimento	
valor	

Modelo lógico

Q4: Buscar as últimas 2 semanas de lançamentos entre conta corrente e cartão de crédito em ordem decrescente de data;

Q4.1: Caso o cliente role a tela, buscar outras duas semanas de lançamentos;

lançamentos_por_período	
id_cliente	K
data_hora	C↓
id	C↑
id_produto	
tipo_lancamento	
status	
descricao	
estabelecimento	
valor	

Modelo lógico

Q5: Buscar os lançamentos do produto e período selecionado pelo cliente;

lançamentos_por_produto	
id_cliente	K
id_produto	C↑
data_hora	C↓
id	C↑
tipo_lancamento	
status	
descricao	
estabelecimento	
valor	

Modelo lógico

Q6: Buscar os lançamentos do tipo (crédito ou débito) e período selecionados pelo cliente;

lancamentos_por_tipo	
id_cliente	K
tipo_lancamento	C↑
data_hora	C↓
id	C↑
id_produto	
status	
descricao	
estabelecimento	
valor	

Modelo lógico

Q7: Buscar o detalhe de um lançamento selecionado pelo cliente;

lancamentos_por_id	
id	K
id_cliente	
tipo_lancamento	
data_hora	
id_produto	
status	
descricao	
estabelecimento	
valor	
codigo_autenticacao	

MODELAGEM NÃO RELACIONAL

Modelo lógico (final)



Modelo físico

MODELAGEM NÃO RELACIONAL

Modelo físico (CQL)

```
1 CREATE KEYSPACE infobarbank
2 WITH replication = {
3     'class': 'SimpleStrategy',
4     'replication_factor': 1
5 };
```

Modelo físico

Q1: Buscar os dados da conta do cliente (apelido do cliente, número da conta, último acesso e saldo disponível);

conta_corrente	
num_conta	text K
id	uuid C↑
apelido_cliente	text
data_ultimo_acesso	timestamp
saldo_disponivel	float

Modelo físico (CQL)

```
9 CREATE TABLE infobarbank.conta_corrente(  
10     num_conta          TEXT,  
11     id                  UUID,  
12     apelido_cliente     TEXT,  
13     data_ultimo_acesso DATE,  
14     saldo_disponivel    FLOAT,  
15     PRIMARY KEY(num_conta, id)  
16 );
```

Modelo físico

Q2: Buscar a lista de cartões de crédito ativos;

cartao_credito	
id_cliente	uuid K
status	text C↑
id	uuid C↑
num_cartao	text
cvv	text
data_validade	date
limite_total	float
limite_disponivel	float

Modelo físico (CQL)

```
27  ✓ CREATE TABLE infobarbank.cartao_credito(  
28      id_cliente          UUID,  
29      status              TEXT,  
30      id                  UUID,  
31      num_cartao          TEXT,  
32      cvv                 TEXT,  
33      data_validade       DATE,  
34      limite_total        FLOAT,  
35      limite_disponivel   FLOAT,  
36      PRIMARY KEY((id_cliente), status, id)  
37  );
```

Modelo físico

Q3: Buscar as últimos **3 lançamentos aprovados** de conta corrente e cartão de crédito em ordem decrescente de data.

Q3.1: Caso o cliente não tenha lançamentos no mês corrente, buscar as três últimas do mês anterior e assim sucessivamente.

Bucketing

Mês de competência
no formato **YYYY-MM**

lançamentos_por_status		
id_cliente	uuid	K
competencia	text	K
status	text	C↑
data_hora	timestamp	C↓
id	uuid	C↑
id_produto	uuid	
tipo_lancamento	text	
descricao	text	
estabelecimento	text	
valor	float	

Modelo físico (CQL)

```
143 CREATE TABLE infobarbank.lancamentos_por_status(  
144     id_cliente      UUID,  
145     competencia     TEXT,  
146     status          TEXT,  
147     data_hora       TIMESTAMP,  
148     id              UUID,  
149     id_produto      UUID,  
150     tipo_lancamento TEXT,  
151     descricao       TEXT,  
152     estabelecimento TEXT,  
153     valor           FLOAT,  
154     PRIMARY KEY((id_cliente, competencia), status, data_hora, id)  
155 ) WITH CLUSTERING ORDER BY (status ASC, data_hora DESC, id ASC) ;
```

Modelo físico

Q4: Buscar as últimas 2 semanas de lançamentos entre conta corrente e cartão de crédito em ordem decrescente de data;

Q4.1: Caso o cliente role a tela, buscar outras duas semanas de lançamentos;

lancamentos_por_periodo		
id_cliente	uuid	K
competencia	text	K
data_hora	timestamp	C↓
id	uuid	C↑
id_produto	uuid	
tipo_lancamento	text	
status	text	
descricao	text	
estabelecimento	text	
valor	float	

Modelo físico (CQL)

```
217  ✓ CREATE TABLE infobarbank.lancamentos_por_periodo(  
218      id_cliente      UUID,  
219      competencia      TEXT,  
220      data_hora        TIMESTAMP,  
221      id               UUID,  
222      status           TEXT,  
223      id_produto       UUID,  
224      tipo_lancamento TEXT,  
225      descricao        TEXT,  
226      estabelecimento TEXT,  
227      valor            FLOAT,  
228      PRIMARY KEY((id_cliente, competencia), data_hora, id)  
229  ) WITH CLUSTERING ORDER BY (data_hora DESC, id ASC) ;
```

Modelo físico

Q5: Buscar os lançamentos do produto e período selecionado pelo cliente;

lançamentos_por_produto		
id_cliente	uuid	K
competencia	text	K
id_produto	uuid	C↑
data_hora	timestamp	C↓
id	uuid	C↑
tipo_lancamento	text	
status	text	
descricao	text	
estabelecimento	text	
valor	float	

Modelo físico (CQL)

```
275 CREATE TABLE infobarbank.lancamentos_por_produto(  
276     id_cliente      UUID,  
277     competencia     TEXT,  
278     id_produto      UUID,  
279     data_hora        TIMESTAMP,  
280     id               UUID,  
281     status           TEXT,  
282     tipo_lancamento TEXT,  
283     descricao        TEXT,  
284     estabelecimento TEXT,  
285     valor            FLOAT,  
286     PRIMARY KEY((id_cliente, competencia), id_produto, data_hora, id)  
287 ) WITH CLUSTERING ORDER BY (id_produto ASC, data_hora DESC, id ASC) ;
```

Modelo físico

Q6: Buscar os lançamentos do tipo (crédito ou débito) e período selecionados pelo cliente;

lançamentos_por_tipo		
id_cliente	uuid	K
competencia	text	K
tipo_lancamento	text	C↑
data_hora	timestamp	C↓
id	uuid	C↑
id_produto	uuid	
status	text	
descricao	text	
estabelecimento	text	
valor	float	

Modelo físico (CQL)

```
353 CREATE TABLE infobarbank.lancamentos_por_tipo(  
354     id_cliente      UUID,  
355     competencia     TEXT,  
356     tipo_lancamento TEXT,  
357     data_hora       TIMESTAMP,  
358     id              UUID,  
359     id_produto       UUID,  
360     status          TEXT,  
361     descricao       TEXT,  
362     estabelecimento TEXT,  
363     valor           FLOAT,  
364     PRIMARY KEY((id_cliente, competencia), tipo_lancamento, data_hora, id)  
365 ) WITH CLUSTERING ORDER BY (tipo_lancamento ASC, data_hora DESC, id ASC) ;
```

Modelo físico

Q7: Buscar o detalhe de um lançamento selecionado pelo cliente;

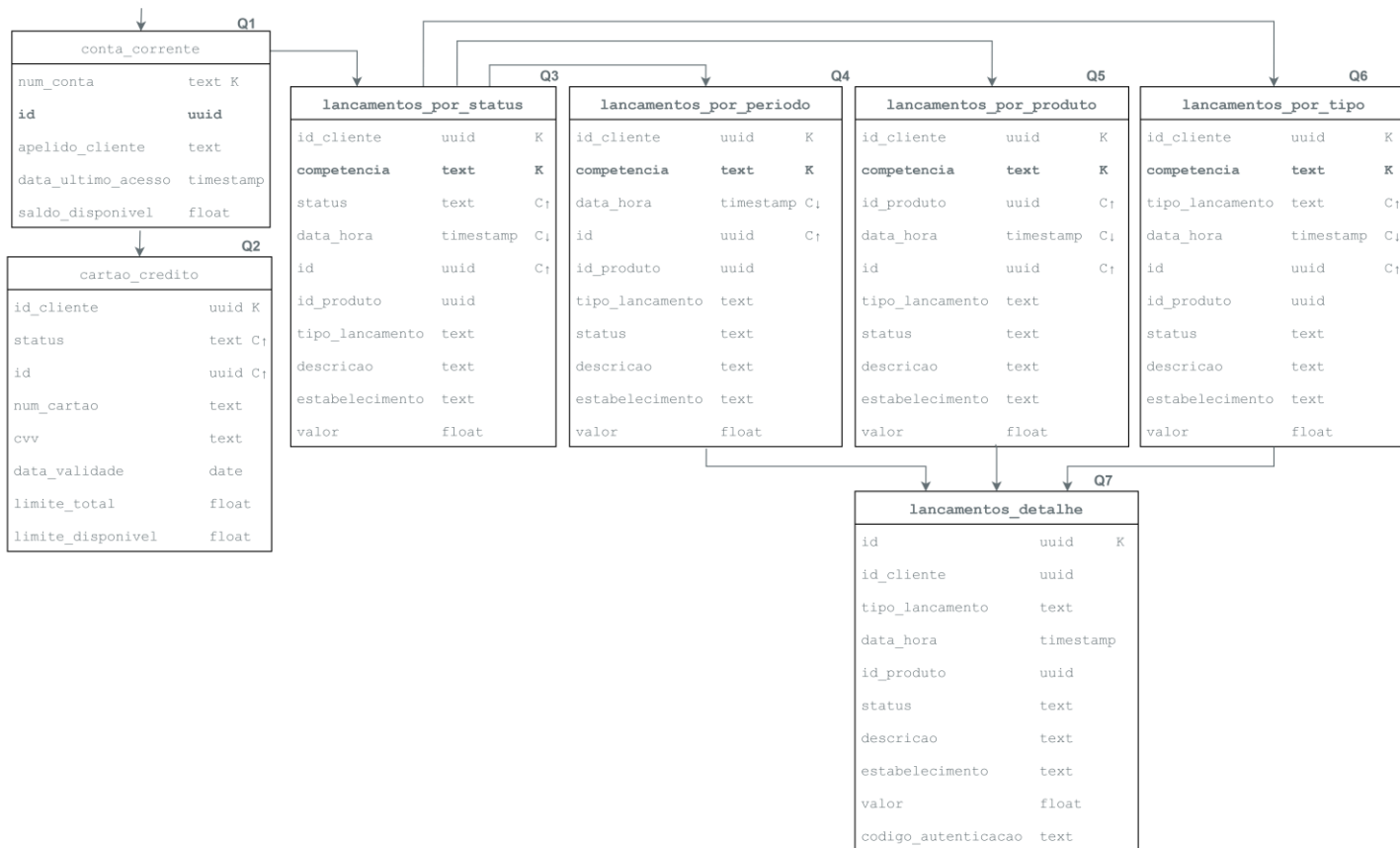
lancamentos_detalle		
id	uuid	K
id_cliente	uuid	
tipo_lancamento	text	
data_hora	timestamp	
id_produto	uuid	
status	text	
descricao	text	
estabelecimento	text	
valor	float	
codigo_autenticacao	text	

Modelo físico (CQL)

```
426 CREATE TABLE infobarbank.lancamentos_detalhe(  
427     id                UUID,  
428     id_cliente        UUID,  
429     competencia       TEXT,  
430     tipo_lancamento  TEXT,  
431     data_hora         TIMESTAMP,  
432     id_produto        UUID,  
433     status            TEXT,  
434     descricao         TEXT,  
435     estabelecimento   TEXT,  
436     valor             FLOAT,  
437     codigo_autenticacao TEXT,  
438     PRIMARY KEY(id)  
439 );
```

MODELAGEM NÃO RELACIONAL

Modelo físico (final)





<https://www.comparably.com/salaries/salaries-for-cassandra-developer>

<https://www.comparably.com/salaries/salaries-for-nosql-database-administrator-cassandra>

<https://www.datastax.com/dev/certifications#details>

OBRIGADO



Copyright © 2019 | Professor (a) Marcelo Barbosa Pinto
Todos os direitos reservados. Reprodução ou divulgação total ou parcial deste documento, é expressamente
proibido sem consentimento formal, por escrito, do professor/autor.

FIAP